Sistemas Operacionais

Sincronização de processos e threads
Parte 1: por que sincronizar?

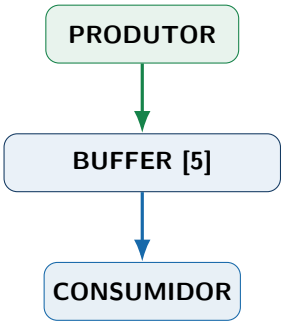
Prof. Frederico Luís Cabral

CEFET/RJ | Ciência da Computação

13 de outubro de 2026

Uma situação real: produzir e consumir dados

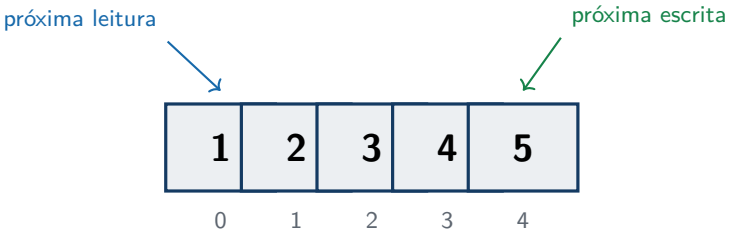
- Uma **thread produtora** gera itens.
- Uma **thread consumidora** processa itens.
- Ambas acessam um **buffer compartilhado**.
- O buffer tem **capacidade finita**: 5 posições.



Pergunta para a turma

O que pode acontecer quando as duas threads executam concorrentemente?

O buffer circular



- Índice de escrita: `entrada = (entrada + 1) % 5`
- Índice de leitura: `saida = (saida + 1) % 5`
- Ao chegar ao fim, cada índice **volta à posição zero**.

Questão

O índice circular, sozinho, impede a sobrescrita de dados não consumidos?

Primeira implementação: dados compartilhados

```c
#define TAM_BUFFER 5
#define TOTAL_ITENS 20

int buffer[TAM_BUFFER] = {0};
int entrada = 0;
int saida = 0;
```

- `buffer`: vetor visível às duas threads.
- `entrada`: posição onde o produtor escreverá.
- `saida`: posição de onde o consumidor lerá.
- Valor `0`: marcador didático de posição vazia (itens válidos: 1 a 20).

O código completo está no arquivo `produtor_consumidor.c` do projeto.

O produtor: escreve sem verificar espaço livre

```
void *produtor(void *arg) {
    for (int item = 1; item <= TOTAL_ITENS; ++item) {
        usleep(100000);           // 100 ms
        int pos = entrada;
        buffer[pos] = item;       // escreve SEM verificar!
        entrada = (entrada + 1) % TAM_BUFFER;
        printf("PRODUZ %d -> posicao %d\n", item, pos);
        mostrar_buffer();
    }
    return NULL;
}
```

Observe

O produtor pode escrever em uma posição que ainda guarda um item não consumido.

O consumidor: lê sem verificar se há item

```c
void *consumidor(void *arg) {
    for (int i = 0; i < TOTAL_ITENS; ++i) {
        usleep(150000);            // 150 ms
        int pos = saida;
        int item = buffer[pos];    // le SEM verificar!
        buffer[pos] = 0;           // limpa visualmente
        saida = (saida + 1) % TAM_BUFFER;
        printf("CONSOME %d <- posicao %d\n", item, pos);
        mostrar_buffer();
    }
    return NULL;
}
```

Observe

O consumidor pode ler uma posição vazia, ou receber um item diferente daquele esperado.

As duas threads executam concorrentemente

```
int main(void) {
    pthread_t tprod, tcons;

    pthread_create(&tprod, NULL, produtor, NULL);
    pthread_create(&tcons, NULL, consumidor, NULL);

    pthread_join(tprod, NULL);
    pthread_join(tcons, NULL);
    return 0;
}
```

- `pthread_create`: inicia cada thread.
- `pthread_join`: faz a thread principal aguardar o término.
- **Nenhuma dessas chamadas protege o buffer.**

Evolução possível do buffer (intercalação ilustrativa)

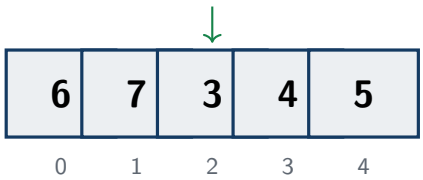
Operação	Buffer após a operação	Observação
Produz 1	[1][−][−][−][−]	escreve na posição 0
Consome 1	[−][−][−][−][−]	retira da posição 0
Produz 2	[−][2][−][−][−]	
Produz 3	[−][2][3][−][−]	
Consome 2	[−][−][3][−][−]	
Produz 4	[−][−][3][4][−]	
Produz 5	[−][−][3][4][5]	
Produz 6	[6][−][3][4][5]	

Esta sequência é ilustrativa: a ordem real varia conforme o escalonamento. Não é uma transcrição de uma execução observada.

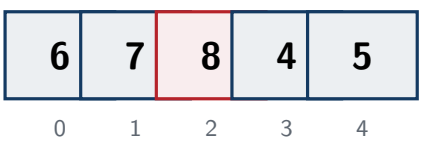
O momento da falha: um item é sobrescrito

O consumidor ainda não leu o item 3.

Produtor escreverá o item 8 na posição 2



O produtor escreve sem esperar por espaço livre:

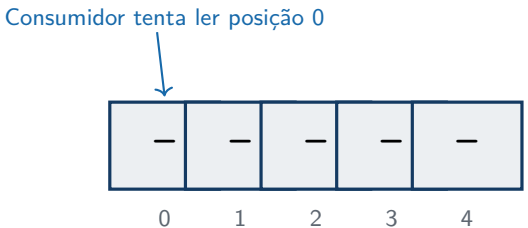


O que aconteceu com o item 3?

O item **8** ocupou a mesma posição: o item **3** foi perdido antes da leitura.

Segundo problema: e se o buffer estiver vazio?

Agora considere o consumidor mais rápido que o produtor.



Pergunta

O que o consumidor recebe se ele lê uma posição que ainda não contém um novo item?

Cenário ilustrativo: pode ocorrer na versão sem sincronização ao alterar as velocidades relativas, por exemplo, consumidor a 50 ms e produtor a 150 ms.

Buffer vazio: o consumidor lê sem esperar

Evento	Estado após o evento
CONSOME 0 ← pos. 0	[-][-][-][-][-]
CONSOME 0 ← pos. 1	[-][-][-][-][-]
PRODUZ 1 → pos. 0	[1][-][-][-][-]

```
int pos = saida;
int item = buffer[pos];   // pode ler 0: posicao vazia!
buffer[pos] = 0;
saida = (saida + 1) % TAM_BUFFER;
```

O defeito

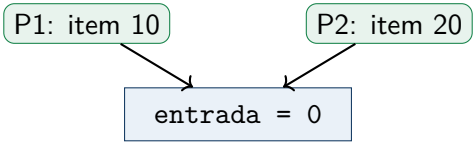
Não existe uma espera por **item disponível**. O consumidor pode avançar a posição de leitura sem ter consumido nenhum item válido.

Sequência possível, não captura de execução. Zero é marcador didático de vazio.

Terceiro problema: condição de corrida no índice

Uma pequena variação do sistema

Agora temos **dois produtores**, P1 e P2, compartilhando a variável global `entrada`. Ambos querem inserir um item.



O que muda?

Diferentemente do caso anterior, **duas threads modificam a mesma variável de posição**, sem exclusão mútua.

O trecho vulnerável: ambos usam `entrada`

```
// Executado tanto por P1 quanto por P2
int pos = entrada;                  // leitura compartilhada
buffer[pos] = item;                 // escrita no buffer
entrada = (entrada + 1) % TAM_BUFFER; // atualizacao
```

Por que isso não é uma única operação?

Uma thread pode ser interrompida entre a leitura de `entrada` e sua atualização. Nesse intervalo, outra thread pode ler o **mesmo valor**.

As duas podem escolher a mesma posição!

Intercalação: duas threads escolhem a posição 0

Passo	Produtor P1 (item 10)	Produtor P2 (item 20)
1	`pos = entrada` (lê 0)	
2		`pos = entrada` (lê 0)
3	`buffer[0] = 10`	
4		`buffer[0] = 20`
5	`entrada = 1`	
6		`entrada = 1`



Resultado desse entrelaçamento

O item **10 foi sobrescrito** por 20, e o índice avançou apenas até 1. Os dois produtores acreditavam ter reservado a mesma posição.

Três falhas, três perguntas

1. **Buffer cheio:** como impedir que o produtor sobrescreva itens pendentes?
2. **Buffer vazio:** como impedir que o consumidor leia sem haver item disponível?
3. **Condição de corrida:** como garantir que dois produtores não escolham a mesma posição?

Duas necessidades distintas

Coordenação (esperar por espaço ou itens) e **exclusão mútua** (proteger atualizações concorrentes).

Em C, acessos conflitantes não atômicos e sem sincronização caracterizam uma data race e geram comportamento indefinido.

Próximos conceitos

1. **Região crítica**: que trechos acessam dados compartilhados?
2. **Exclusão mútua**: como impedir interferências perigosas?
3. **Coordenação**: como aguardar *buffer não vazio* ou *buffer não cheio*?
4. **Primitivas**: mutex, variáveis de condição e semáforos.

Mensagem principal

Ter duas threads em execução não garante que elas cooperem corretamente. **É necessário sincronizar.**

Região Crítica

Definição

Uma **região crítica** é um trecho de código que acessa recursos compartilhados e cuja execução concorrente precisa ser controlada para preservar a correção do programa.

Exemplos de recursos compartilhados:

- Variáveis e estruturas de dados em memória;
- Buffers e filas compartilhadas;
- Arquivos e dispositivos de entrada e saída;
- Estruturas internas do Sistema Operacional.

Problema

A execução concorrente de operações sobre um recurso compartilhado pode produzir resultados incorretos quando não existe sincronização adequada.

Região Crítica: Produtor-Consumidor

Produtor

```
int posicao = entrada;

buffer[posicao] = item;

entrada = (entrada + 1)
          % TAM_BUFFER;
```

Consumidor

```
int posicao = saida;

int item = buffer[posicao];

buffer[posicao] = 0;

saida = (saida + 1)
        % TAM_BUFFER;
```

O que existe em comum?

Ambas as threads acessam o mesmo `buffer`.
Se acessarem simultaneamente a mesma posição, poderão ocorrer interferências entre suas operações.

Exclusão Mútua

Definição

A **exclusão mútua** é uma propriedade de sincronização que garante que, em um determinado instante, **no máximo uma thread ou processo** execute uma região crítica protegida pelo mesmo mecanismo.

Objetivo:

- Evitar acessos conflitantes simultâneos;
- Preservar a consistência dos dados compartilhados;
- Impedir interferências entre operações críticas.

Ideia fundamental

Se uma thread está executando a região crítica, outra thread que deseja executar a mesma região protegida deve aguardar.

Estrutura de uma Solução com Exclusão Mútua

Estrutura geral

```
while (1) {

    /* Regiao de entrada */
    /* REGIAO CRITICA */
    /* Regiao de saida */
    /* Regiao restante */

}
```

- **Região de entrada:** solicita permissão para acessar a região crítica.
- **Região crítica:** acessa ou modifica os recursos compartilhados.
- **Região de saída:** libera a região crítica.
- **Região restante:** executa operações que não precisam daquela exclusão mútua.

Exclusão Mútua é Suficiente?

Considere novamente o problema do produtor-consumidor.

Mesmo que produtor e consumidor nunca acessem simultaneamente o buffer:

- O produtor ainda pode tentar inserir um item quando o buffer estiver **cheio**.
- O consumidor ainda pode tentar retirar um item quando o buffer estiver **vazio**.

Conclusão

Exclusão mútua não é suficiente!
Além de controlar o acesso concorrente, precisamos **coordenar a execução** das threads de acordo com o estado do buffer.

Taxonomia das Soluções de Exclusão Mútua

Critério 1: Como a thread espera?

- **Espera ocupada (busy waiting):** repete testes usando CPU.
- **Espera com bloqueio:** suspende a execução até poder prosseguir.

Critério 2: Qual é o mecanismo empregado?

- **Algoritmos clássicos por software:** alternância estrita e Peterson.
- **Primitivas atômicas:** Test-and-Set, Compare-and-Swap.
- **Abstrações de biblioteca/SO:** mutexes e outras primitivas.

Atenção

Os dois critérios são **independentes**: um mutex pode usar operações atômicas e, dependendo da implementação, bloquear a thread.

Primeira Tentativa: Variável de Controle

Considere uma variável compartilhada:

Pseudocódigo

```
int ocupado = 0;
while (ocupado == 1);
ocupado = 1;
/* Regiao critica */
ocupado = 0;
```

- `ocupado = 0`: região crítica livre.
- `ocupado = 1`: região crítica ocupada.
- A thread espera enquanto a região estiver ocupada.

Pergunta

Essa solução realmente garante exclusão mútua?

Por Que a Primeira Tentativa Falha?

Suponha inicialmente:

$$ocupado = 0$$

Passo	Thread 1	Thread 2
1	Lê ocupado = 0	
2		Lê ocupado = 0
3	ocupado = 1	
4		ocupado = 1
5	Entra na região crítica	
6		Entra na região crítica

Falha

As duas threads entram na região crítica!
A verificação e a atualização da variável ocupado não formam uma operação atômica.

Espera Ocupada (Busy Waiting)

Definição

A thread verifica repetidamente uma condição para tentar entrar na região crítica, sem se bloquear voluntariamente.

Exemplo conceitual

```
while (ocupado)
    ;  /* continua testando */
```

- Consome tempo de CPU durante a espera.
- Aparece na primeira tentativa, na alternância estrita, em Peterson e nos spinlocks.
- **Não implica, por si só, incorreção:** Peterson é correto sob as hipóteses do modelo teórico.

Requisitos de uma Solução Correta

Uma solução clássica para o problema da região crítica deve satisfazer três propriedades.

1. Exclusão Mútua

No máximo uma thread pode executar a região crítica em determinado instante.

2. Progresso

Se nenhuma thread está na região crítica, a escolha de quem entrará não pode ser adiada indefinidamente pelas threads que não desejam entrar.

3. Espera Limitada

Após uma thread solicitar entrada, deve existir um limite para o número de entradas das outras threads antes dela.

Segunda Tentativa: Alternância Estrita

Considere duas threads identificadas por `0` e `1`.

Variável compartilhada

```
int vez = 0;
```

Thread 0

```
while (vez != 0);
    /* Regiao critica */
vez = 1;
```

Thread 1

```
while (vez != 1);
    /* Regiao critica */
vez = 0;
```

Ideia

As threads recebem permissão para entrar na região crítica alternadamente, com **espera ocupada**.

O Problema da Alternância Estrita

Suponha que:

$$vez = 0$$

1. A Thread 0 entra na região crítica.
2. Ao sair, executa `vez = 1`.
3. A Thread 1 não deseja entrar na região crítica.
4. A Thread 0 tenta entrar novamente.
5. A Thread 0 permanece esperando, pois `vez = 1`.

Problema

A Thread 0 fica impedida de entrar, mesmo com a região crítica livre.

A solução garante exclusão mútua, mas não satisfaz o requisito de progresso.

Algoritmo de Peterson

O algoritmo de Peterson utiliza duas informações:

- Se uma thread deseja entrar na região crítica;
- Qual thread recebe prioridade em caso de disputa.

Variáveis compartilhadas

```
bool interesse[2] = {false, false};

int vez = 0;
```

- `interesse[i]` indica que a thread `i` deseja entrar.
- `vez` é utilizada para resolver a disputa quando ambas desejam entrar.

Algoritmo de Peterson: Pseudocódigo

Para a thread `i`, com $j = 1 - i$:

Algoritmo teórico

```
interesse[i] = true;
vez = j;
while (interesse[j] && vez == j)
    ;
/* Regiao critica */
interesse[i] = false;
```

- A thread anuncia seu interesse e cede prioridade à outra.
- O laço `while` é uma forma de **espera ocupada**.
- Ao sair, a thread retira seu interesse.

Por Que Peterson Funciona?

Considere duas threads tentando entrar simultaneamente.

1. Ambas indicam interesse em entrar.
2. Ambas atualizam a variável vez.
3. A última atualização determina qual thread deverá ceder prioridade.
4. Uma thread entra na região crítica.
5. A outra aguarda até que a primeira retire seu interesse.

Resultado

No modelo clássico de memória sequencialmente consistente, Peterson satisfaz exclusão mútua, progresso e espera limitada para duas threads.

Das Soluções Clássicas ao OpenMP

O algoritmo de Peterson demonstra como garantir exclusão mútua utilizando variáveis compartilhadas.

Entretanto, sua utilização prática apresenta limitações:

- A versão estudada funciona com duas threads.
- Utiliza espera ocupada.
- Depende de garantias de atomicidade e ordenação.
- Exige implementar o protocolo de sincronização.

Pergunta

Precisamos implementar algoritmos como Peterson sempre que quisermos proteger uma região crítica?

Introdução ao OpenMP

Definição

OpenMP é uma API para programação paralela em memória compartilhada, utilizada com C, C++ e Fortran.

Características:

- Permite criar regiões de execução paralela.
- Utiliza threads que compartilham memória.
- Oferece diretivas para distribuir trabalho.
- Disponibiliza mecanismos de sincronização.

Diretivas

Em C, as diretivas OpenMP são introduzidas por `#pragma omp`.

OpenMP: Primeiro Programa

```
#include <stdio.h>
#include <omp.h>
int main(void)
{
    #pragma omp parallel num_threads(4)
    {
        int id = omp_get_thread_num();
        printf("Thread %d\n", id);
    }
    return 0;
}
```

Compilação e execução

```
gcc -fopenmp exemplo.c -o exemplo
./exemplo
```

- `parallel`: cria uma região paralela.

Como as Threads Executam?

Uma possível saída

```
Thread 0
Thread 2
Thread 1
Thread 3
```

- Todas as threads executam o bloco delimitado pela região `parallel`.
- A ordem das mensagens não é determinada.
- As threads podem executar simultaneamente em diferentes núcleos.

Importante

A existência de múltiplas threads não garante uma ordem específica de execução.

Variáveis Compartilhadas e Privadas

```
int compartilhada = 0;
#pragma omp parallel num_threads(4)
{
    int privada = omp_get_thread_num();
    #pragma omp critical
    {
        compartilhada++;
    }
}
```

Variável compartilhada

`compartilhada` é acessada pelas quatro threads.

Variável privada

Cada thread possui sua própria variável local `privada`.

Condição de Corrida com OpenMP

Considere o seguinte programa:

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    contador++;
}
printf("Contador = %ld\n", contador);
```

Resultado esperado:

$$contador = 1.000.000$$

Problema

As quatro threads modificam a mesma variável `contador` sem sincronização.
A operação `contador++` não é atômica.

OpenMP: Diretiva Critical

A diretiva `critical` permite proteger uma região crítica.

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    #pragma omp critical
    {
        contador++;
    }
}
```

Funcionamento

Apenas uma thread por vez pode executar a região `critical` correspondente.

Resultado

O incremento passa a ser protegido por exclusão mútua.

Como Funciona a Diretiva Critical?

Considere quatro threads tentando executar a mesma região crítica.

Thread	Situação
Thread 0	Executando a região crítica
Thread 1	Aguardando
Thread 2	Aguardando
Thread 3	Aguardando

Quando a Thread 0 termina, outra thread pode entrar na região crítica.

Propriedade garantida

No máximo uma thread executa a mesma região crítica protegida em determinado instante.

Observação

A diretiva não estabelece uma ordem específica para a entrada das threads.

OpenMP: Diretiva Atomic

Quando precisamos proteger uma atualização simples, podemos utilizar a diretiva `atomic`.

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    #pragma omp atomic
    contador++;
}
```

Funcionamento

A diretiva garante a atomicidade da atualização de `contador` em relação a acessos atômicos compatíveis.

- É aplicável a operações específicas.
- Pode ser mais eficiente que `critical`.
- Não protege arbitrariamente um bloco de código.

OpenMP: Diretiva Reduction

Outra possibilidade é evitar que todas as threads atualizem diretamente o mesmo contador.

```
long contador = 0;
#pragma omp parallel for reduction(+:contador)
for (int i = 0; i < 1000000; i++) {
    contador++;
}
```

Funcionamento

Cada thread trabalha com uma cópia privada da variável de redução.
Ao final, os resultados são combinados utilizando a operação especificada.

Diferença fundamental

A redução evita a disputa direta pelo contador durante as iterações, em vez de proteger cada incremento com exclusão mútua.

Comparando as Soluções em OpenMP

Solução	**Funcionamento**	**Aplicação**
`critical`	Exclusão mútua entre regiões correspondentes	Proteção de blocos de código
`atomic`	Atomicidade de operações específicas	Atualizações simples de variáveis
`reduction`	Cópias privadas e combinação dos resultados	Somas, contagens e outras reduções

Conclusão

Nem toda sincronização precisa ser implementada por meio de uma região crítica explícita.
A escolha do mecanismo depende da operação e dos requisitos do programa.